The CN-OBEE dataset, occupant behavior and appliance electricity use of a single-family apartment in China

This notebook provides a step-by-step guide on how to (1) load and analyse the dataset, (2) visualise missing data for each room, and finally (3) technical validation of CN-OBEE.

Building Description

This dataset is based on a duplex apartment located in Miyun District, Beijing. The apartment is on the middle floor of the building, with a building area of 172.73 square meters. There is a master bedroom, a secondary bedroom, a cloakroom,a home office, a living room, a kitchen and two bathrooms in the residence.

Dataset Description

The CN-OBEE dataset consists of eight separate CSV files.Six files contain the data collected from 6 rooms and consist of five different data categories, including indoor environment (temperature, humidity and air pressure), occupancy state, window-opening state.The other twe files are power of 6 air conditioners and 8 household appliances and weather file. Six air conditioners are respectively located in 6 room. Eight household appliances are computer, TV, electric kettle, rice cooker, fridge, washing machine and water heaters upstairs and downstairs.

1. Load and Analyse the Dataset

Download the dataset from the figshare link provided. Then, import the necessary libraries and load the dataset.

In [1]:
```
# import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib as mpl
from matplotlib import style
from matplotlib.ticker import LinearLocator
from matplotlib.ticker import MaxNLocator
from matplotlib.ticker import MultipleLocator
import missingno as msno
import seaborn as sns
import os.path
import sys
import datetime
from copy import deepcopy
from mpl_toolkits.axes_grid1.inset_locator import zoomed_inset_axes, mark_inset, inset_axes
from matplotlib import rcParams
import matplotlib.ticker as ticker
from mpl_toolkits.axes_grid.parasite_axes import SubplotHost
import warnings
warnings.filterwarnings("ignore")

mpl.rcParams['agg.path.chunksize'] = 10000
plt.rcParams['font.sans-serif']=['Arial']
plt.rcParams['axes.unicode_minus']=False
```

```
<ipython-input-1-c46cedaa2051>:20: MatplotlibDeprecationWarning:
The mpl_toolkits.axes_grid module was deprecated in Matplotlib 2.1 and will be removed two minor releases later. Use mpl_toolkits.axes_grid1 and mpl_toolkits.axisartist, which provide the same functionality instead.
  from mpl_toolkits.axes_grid.parasite_axes import SubplotHost
```

In [2]:
```
# load the ROBOD dataset for each room
csv_names = ["Master_bedroom", "Secondary_bedroom", "Cloakroom", "Home_office",
             "Living_room", "Kitchen", "Power", "Outdoor_weather"]
data_dict = {}
for csv in csv_names:
    data_dict[csv] = {}
    data_dict[csv]["raw"] = pd.read_csv("E:/DDOBE/{}.csv".format(csv))
    #print("Generating information for {}".format(csv))
    #print(data_dict[csv]["raw"].info())
    #print()
```

In [3]:
```
overall_fontsize = 15

nice_fonts = {"axes.titlesize": overall_fontsize,
              "axes.labelsize": overall_fontsize,
              "font.size": overall_fontsize,
              "legend.fontsize": overall_fontsize,
              "xtick.labelsize": overall_fontsize,
              "ytick.labelsize": overall_fontsize}
mpl.rcParams.update(nice_fonts)
```

2. Visualisations of Missing Data

In [4]:

```
def missing_data_graph():
    data1 = data_dict["Master_bedroom"]["raw"]
    data2 = data_dict["Secondary_bedroom"]["raw"]
    data3 = data_dict["Cloakroom"]["raw"]
    data4 = data_dict["Home_office"]["raw"]
    data5 = data_dict["Living_room"]["raw"]
    data6 = data_dict["Kitchen"]["raw"]
    data7 = data_dict["Power"]["raw"]

    data1["air conditioner"] = data7["air conditioner#1"]
    data2["air conditioner"] = data7["air conditioner#2"]
    data3["air conditioner"] = data7["air conditioner#3"]
    data4["air conditioner"] = data7["air conditioner#4"]
    data5["air conditioner"] = data7["air conditioner#5"]
    data6["air conditioner"] = data7["air conditioner#6"]

    data4["computer"] = data7["computer"]
    data5["electric kettle"] = data7["electric kettle"]
    data5["TV"] = data7["TV"]
    data6["fridge"] = data7["fridge"]
    data6["rice cooker"] = data7["rice cooker"]
    data6["upstairs water heater"] = data7["upstairs water heater"]
    data6["downstairs water heater"] = data7["downstairs water heater"]
    data6["washing machine"] = data7["washing machine"]

    data = pd.merge(data1, data2, on='time')
    data = pd.merge(data, data3, on='time')
    data = pd.merge(data, data4, on='time')
    data = pd.merge(data, data5, on='time')
    data = pd.merge(data, data6, on='time')

    DATA_COLUMNS = ["time","temperature_1","humidity_1","air pressure_1","window state_1","occupancy state
                    "temperature_2","humidity_2","air pressure_2","window state_2","occupancy state_2","ac
                    "temperature_3","humidity_3","air pressure_3","occupancy state_3","ac_3",
                    "temperature_4","humidity_4","air pressure_4","west window state_4","east window state
                    "temperature_5","humidity_5","air pressure_5","west window state_5","east window state
                    "temperature_6","humidity_6","air pressure_6","window state_6","occupancy state_6","ac
                    "upstairs water heater","downstairs water heater","washing machine"]
    data.columns = DATA_COLUMNS
    rooms_mapping = {"Master_bedroom":"master bedroom",
                     "Secondary_bedroom":"secondary bedroom",
                     "Cloakroom":"cloakroom",
                     "Home_office":"home office",
                     "Living_room":"living room",
                     "Kitchen":"kitchen"}

    fig, ax1 = plt.subplots(1,1, figsize=(20,10), dpi=800)

    raw_data1 = deepcopy(data)
    raw_data1[["date","time"]] = raw_data1["time"].astype(str).str.split(' ',expand=True)

    raw_data2 = deepcopy(data)
    raw_data2['occupancy state_1'] = raw_data2['occupancy state_1'].fillna(0)
    raw_data2['occupancy state_2'] = raw_data2['occupancy state_2'].fillna(0)
    raw_data2['occupancy state_3'] = raw_data2['occupancy state_3'].fillna(0)
    raw_data2['occupancy state_4'] = raw_data2['occupancy state_4'].fillna(0)
    raw_data2['occupancy state_5'] = raw_data2['occupancy state_5'].fillna(0)
    raw_data2['occupancy state_6'] = raw_data2['occupancy state_6'].fillna(0)
    COLUMNS = ["temperature_1","humidity_1","air pressure_1","window state_1","occupancy state_1","ac_1",
               "temperature_2","humidity_2","air pressure_2","window state_2","occupancy state_2","ac_2",
               "temperature_3","humidity_3","air pressure_3","occupancy state_3","ac_3",
               "temperature_4","humidity_4","air pressure_4","west window state_4","east window state_4","
               "temperature_5","humidity_5","air pressure_5","west window state_5","east window state_5","
               "temperature_6","humidity_6","air pressure_6","window state_6","occupancy state_6","ac_6","
               "upstairs water heater","downstairs water heater","washing machine"]
    raw_data2 = raw_data2[COLUMNS]

    #print(raw_data2)

    msno.matrix(raw_data2, ax = ax1, sparkline=False)
    #ax1.set_xticklabels(labels=list(raw_data2.columns))
    month = [1440*1,1440*31,1440*62,1440*93,1440*123,1440*154,1440*184,1440*215,1440*246,1440*274,1440*3
    ax1.set_yticks(month)
    ax1.set_yticklabels([raw_data1.loc[i,"date"] for i in month],fontsize=15)

    plt.savefig('missing.jpg', dpi=800, bbox_inches="tight",facecolor='w')
```

```
    #plt.show()
    plt.close()

missing_data_graph()
```

3. Technical Validation Visualisations

1. Indoor and outdoor thermal environment

In [5]:

```
def environment_graph():
    for csv in csv_names:
        data_dict[csv]["raw"]["time"] = pd.to_datetime(data_dict[csv]["raw"]["time"])
    data1 = data_dict["Master_bedroom"]["raw"]
    data2 = data_dict["Secondary_bedroom"]["raw"]
    data3 = data_dict["Cloakroom"]["raw"]
    data4 = data_dict["Home_office"]["raw"]
    data5 = data_dict["Living_room"]["raw"]
    data6 = data_dict["Kitchen"]["raw"]
    data7 = data_dict["Outdoor_weather"]["raw"]

    f, (ax1,ax2,ax3) = plt.subplots(1,3,figsize = (16,6), dpi=800)

    df1 = pd.concat([data1["dry-bulb temperature (° C)"],data2["dry-bulb temperature (° C)"],data3["dry-bu
                     data4["dry-bulb temperature (° C)"],data5["dry-bulb temperature (° C)"],data6["dry-bu
                     data7["dry-bulb temperature (° C)"]],axis = 1)
    df1.columns =["room1","room2","room3","room4","room5","room6","outdoor"]
    #print(df1)
    sns.boxplot(data=df1,ax=ax1)
    ax1.set_xticklabels(["Master\n bedroom","Secondary\n bedroom", "Cloakroom", "Home office",
                         "Living room", "Kitchen","Outdoor"],rotation=45)
    ax1.set_ylabel('Dry-bulb temperature (° C)')

    df2 = pd.concat([data1["relative humidity (%)"],data2["relative humidity (%)"],data3["relative humidit
                     data4["relative humidity (%)"],data5["relative humidity (%)"],data6["relative humidit
                     data7["relative humidity (%)"]],axis = 1)
    df2.columns =["room1","room2","room3","room4","room5","room6","outdoor"]
    #print(df2)
    sns.boxplot(data=df2,ax=ax2)
    ax2.set_xticklabels(["Master\n bedroom","Secondary\n bedroom", "Cloakroom", "Home office",
                         "Living room", "Kitchen","Outdoor"],rotation=45)
    ax2.set_ylabel('Relative humidity (%)')

    df3 = pd.concat([data1["air pressure (Pa)"]-101325,data2["air pressure (Pa)"]-101325,data3["air pressu
                     data4["air pressure (Pa)"]-101325,data5["air pressure (Pa)"]-101325,data6["air pressu
                     data7["atmospheric pressure (hPa)"]*100-101325],axis = 1)
    df3.columns =["room1","room2","room3","room4","room5","room6","outdoor"]
    #print(df3)
    sns.boxplot(data=df3,ax=ax3)
    ax3.set_xticklabels(["Master\n bedroom","Secondary\n bedroom", "Cloakroom", "Home office",
                         "Living room", "Kitchen","Outdoor"],rotation=45)
    ax3.set_ylabel('Relative pressure (Pa)')

    plt.tight_layout()
    plt.savefig('environmennt.jpg', dpi=800, bbox_inches="tight",facecolor='w')
    #plt.show()
    plt.close()

environment_graph()
```

2. Occupancy presence

In [6]:

```
def occupancy_graph():
    fig, ax1 = plt.subplots(1,1,figsize = (16, 4), dpi=800)

    occu = pd.DataFrame()
    for i in range(6):
        room = csv_names[i]
        data = deepcopy(data_dict[room]["raw"])
        data.set_index(["time"],inplace=True)
        data.index = pd.to_datetime(data.index)
        occu[room] = data['occupancy presence (0/1)']

    occu['Row_sum'] = occu.apply(lambda x: x.sum(), axis=1)
    #occu = occu[(occu.index >= pd.to_datetime('20211215')) & (occu.index <= pd.to_datetime('20211231'))]
    ax1.plot(occu.index, occu['Row_sum'], linewidth=1, color="grey", label="Detected")
    a = (occu > 3).sum()
    b = len(occu)
    #print(a)
    #print(b)
    print(a/b)
    c = [3] * 527041
    ax1.plot(occu.index,c,linewidth=2, color="red", label="Reference")
    ax1.set_xlabel("Time")
    ax1.set_ylabel("Number of motions")
    ax1.set_yticks(range(0,5,1))
    plt.legend(loc="upper right")
    plt.tight_layout()
    plt.savefig('occupancy.jpg', dpi=800, bbox_inches="tight",facecolor='w')
    #plt.show()
    plt.close()

occupancy_graph()
```

```
Master_bedroom      0.000000
Secondary_bedroom   0.000000
Cloakroom           0.000000
Home_office         0.000000
Living_room         0.000000
Kitchen             0.000000
Row_sum             0.000241
dtype: float64
```

In [7]:

```
colour_mapping = {"Master_bedroom":"#1f77b4",
                  "Secondary_bedroom":"#ff7f0e",
                  "Cloakroom":"#2ca02c",
                  "Home_office":"#d62728",
                  "Living_room":"#FFB6C1",
                  "Kitchen":"#9467bd"}
rooms_mapping = {"Master_bedroom":"Master bedroom",
                 "Secondary_bedroom":"Secondary bedroom",
                 "Cloakroom":"Cloakroom",
                 "Home_office":"Home office",
                 "Living_room":"Living room",
                 "Kitchen":"Kitchen"}
name_1 = ["Master_bedroom", "Cloakroom", "Living_room", "Kitchen"]
name_2 = ["Secondary_bedroom", "Home_office"]
def occupant_graph():
    fig, axes = plt.subplots(6,1,sharex=True,figsize=(16,12),dpi=800)
    for i in range(4):
        room = name_1[i]
        data = deepcopy(data_dict[room]["raw"])
        data.set_index(["time"],inplace=True)
        data.index = pd.to_datetime(data.index)
        data = data['occupancy presence (0/1)']
        data = data.resample('1h').max().ffill()
        #print(data)
        data = data.reset_index()
        #print(data)
        data[["date","t"]] = data["time"].astype(str).str.split(' ',expand=True)
        data[["hour","minute","second"]] = data["t"].astype(str).str.split(':',expand=True)
        data["time"] = pd.to_datetime(data["time"])
        data['week'] = data["time"].dt.dayofweek

        df1 = data[data['week'].isin([0,1,2,3,4])]
        df1 = df1.groupby(by=['hour'])['occupancy presence (0/1)'].mean().reset_index()
        #print(df1)
        axes[i].plot(df1["hour"], df1["occupancy presence (0/1)"], linewidth=1, color=colour_mapping[room]

        df2 = data[data['week'].isin([5,6])]
        df2 = df2.groupby(by=['hour'])['occupancy presence (0/1)'].mean().reset_index()
        #print(df2)
        axes[i].plot(df2["hour"], df2["occupancy presence (0/1)"], linewidth=2, color=colour_mapping[room]

        axes[i].legend(loc="upper left")
        #axes[i].set_ylabel('Probability')
        axes[i].set_ylim(0,1)
        axes[i].yaxis.set_major_locator(LinearLocator(5))
        axes[i].yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1, decimals=0))
        axes[i].set_title(rooms_mapping[room])

        axes[i].set_xticks ([0,3,6,9,12,15,18,21])
        axes[i].set_xticklabels(['00:00','03:00','06:00','09:00','12:00','15:00','18:00','21:00'])
    #axes[3].set_xlabel("(a) There is a significant deviation in the peak on weekdays and weekends.")

    for i in range(2):
        room = name_2[i]
        data = deepcopy(data_dict[room]["raw"])
        data.set_index(["time"],inplace=True)
        data.index = pd.to_datetime(data.index)
        data = data['occupancy presence (0/1)']
        data = data.resample('1h').max().ffill()
        #print(data)
        data = data.reset_index()
        #print(data)
        data[["date","t"]] = data["time"].astype(str).str.split(' ',expand=True)
        data[["hour","minute","second"]] = data["t"].astype(str).str.split(':',expand=True)
        data["time"] = pd.to_datetime(data["time"])
        data['week'] = data["time"].dt.dayofweek

        df1 = data[data['week'].isin([0,1,2,3,4])]
        df1 = df1.groupby(by=['hour'])['occupancy presence (0/1)'].mean().reset_index()
        #print(df1)
        axes[i+4].plot(df1["hour"], df1["occupancy presence (0/1)"], linewidth=1, color=colour_mapping[roo

        df2 = data[data['week'].isin([5,6])]
        df2 = df2.groupby(by=['hour'])['occupancy presence (0/1)'].mean().reset_index()
        #print(df2)
        axes[i+4].plot(df2["hour"], df2["occupancy presence (0/1)"], linewidth=2, color=colour_mapping[roo
```

```
77
78          axes[i+4].legend(loc="upper left")
79          #axes[i].set_ylabel('Probability')
80          axes[i+4].set_ylim(0,0.5)
81          axes[i+4].yaxis.set_major_locator(LinearLocator(3))
82          axes[i+4].yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1, decimals=0))
83          axes[i+4].yaxis.set_minor_locator(LinearLocator(11))
84          axes[i+4].yaxis.set_minor_formatter(plt.NullFormatter())
85          axes[i+4].set_title(rooms_mapping[room])
86
87          axes[i+4].set_xticks ([0,3,6,9,12,15,18,21])
88          axes[i+4].set_xticklabels(['00:00','03:00','06:00','09:00','12:00','15:00','18:00','21:00'])
89      #axes[5].set_xlabel("(b) There is a difference of occupant presence probability on weekdays and weeken
90
91      fig.text(0.5, 0, 'Time', ha='center')
92      fig.text(0, 0.5, 'Occupancy probability', ha='center', va='center', rotation='vertical')
93      plt.tight_layout()
94      plt.savefig('occupant.jpg', dpi=800, bbox_inches="tight",facecolor='w')
95      #plt.show()
96      plt.close()
97
98  occupant_graph()
```

3. Relationship between window-opening and occupancy

In [8]:

```
def windows_occupant_graph1():
    room_names = ["Master_bedroom", "Secondary_bedroom", "Home_office",
                  "Living_room", "Kitchen"]

    df1 = pd.read_csv("E:/lxy/我的坚果云/data/原始数据/('客厅门窗传感器东', 'magnet_status').csv".format(c
    df1.set_index(["t"],inplace=True)
    df1.index = pd.to_datetime(df1.index)
    df1 = df1['value']
    df1 = df1.resample('1T').max()
    df2 = pd.read_csv("E:/lxy/我的坚果云/data/原始数据/('客厅门窗传感器西', 'magnet_status').csv".format(c
    df2.set_index(["t"],inplace=True)
    df2.index = pd.to_datetime(df2.index)
    df2 = df2['value']
    df2 = df2.resample('1T').max()
    win = pd.DataFrame()
    win["east"] = df1
    win["west"] = df2
    win.replace("0","1",inplace = True)
    win['Row_sum'] = win.apply(lambda x: x.sum(), axis=1)
    #win = win['Row_sum']
    win = win.resample('15T').sum()
    win["minOfDay"] = win.index.minute
    win["hourOfDay"] = win.index.hour
    win["Day"] = win["hourOfDay"]*60 + win["minOfDay"]
    win = win[(win.index >= pd.to_datetime('20210531')) & (win.index <= pd.to_datetime('20220601'))]
    date_range = pd.date_range(start='2021-5-31', periods=96, freq='15T')
    win = win.groupby(by=["Day"])['Row_sum'].sum().reset_index()
    win.index = date_range

    data1 = deepcopy(data_dict["Living_room"]["raw"])
    data1.set_index(["time"],inplace=True)
    data1.index = pd.to_datetime(data1.index)
    data1["minOfDay"] = (data1.index.hour)*60+(data1.index.minute)
    date1_range = pd.date_range(start='2021-5-31', periods=1440, freq='1T')
    data1 = data1.groupby(by=["minOfDay"])['occupancy presence (0/1)'].mean().reset_index()
    data1.index = date1_range

    fig, ax1 = plt.subplots(1,1,figsize = (16, 4), dpi=800)

    l1, = ax1.plot(data1.index,data1["occupancy presence (0/1)"],linewidth=1,color="red")
    ax2 = ax1.twinx()
    l2, = ax2.plot(win.index, win["Row_sum"], linewidth=1, color="grey")
    hours = mdates.HourLocator(interval = 4)
    h_fmt = mdates.DateFormatter('%H:%M')
    ax1.xaxis.set_major_locator(hours)
    ax1.xaxis.set_major_formatter(h_fmt)
    ax1.set_xlabel("Time")
    ax1.set_ylabel("Occupancy probability")
    ax1.yaxis.set_major_locator(MaxNLocator(8,integer=True))
    ax1.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1, decimals=0))
    ax2.set_ylabel("Total number of\n"+"window opening and closing")
    plt.legend(handles=[l1, l2], labels=["Living_room occupancy", "Window opening and closing"], loc=0)
    #plt.legend(loc="upper right")
    plt.tight_layout()

    plt.savefig('windows_occupant2.jpg', dpi=800, bbox_inches="tight",facecolor='w')
    #plt.show()
    plt.close()

windows_occupant_graph1()
```

4. Relations between appliances power and occupancy

In [9]:

```
def occupancy_power_graph():
    fig, ax1 = plt.subplots(1,1,figsize = (16, 4), dpi=800)
    data1 = deepcopy(data_dict["Master_bedroom"]["raw"])
    data1.set_index(["time"],inplace=True)
    data1.index = pd.to_datetime(data1.index)
    data1["minOfDay"] = (data1.index.hour)*60+(data1.index.minute)
    df1 = data1.groupby(by=['minOfDay'])['occupancy presence (0/1)'].mean().reset_index()
    #print(df1)

    data2 = deepcopy(data_dict["Power"]["raw"])
    data2.set_index(["time"],inplace=True)
    data2.index = pd.to_datetime(data2.index)
    data2["minOfDay"] = (data2.index.hour)*60+(data2.index.minute)
    df2 = data2.groupby(by=['minOfDay'])['upstairs water heater'].mean().reset_index()
    #print(df2)

    l1, = ax1.plot(df1["minOfDay"], df1["occupancy presence (0/1)"], linewidth=1, color="red")
    ax1.yaxis.set_major_locator(MaxNLocator(8,integer=True))
    ax1.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1, decimals=0))
    ax1.set_ylabel('Occupancy probability')
    ax1.set_xlabel('Time')
    ax2 = ax1.twinx()
    l2, = ax2.plot(df2["minOfDay"],df2["upstairs water heater"],linewidth=1,color="grey")
    #ax2.set_xlabel('Time')
    ax2.set_ylabel('Average power (W)')
    ax2.set_xlim(0,1439)
    ax2.set_xticks ([0,180,360,540,720,900,1080,1260])
    ax2.set_xticklabels(['00:00','03:00','06:00','09:00','12:00','15:00','18:00','21:00'])
    plt.legend(handles=[l1, l2], labels=['Master_bedroom occupancy', 'Upstairs water heater'], loc=0)
    plt.savefig('occupancy and power_1.jpg', dpi=800, bbox_inches="tight",facecolor='w')
    #plt.show()
    plt.close()
occupancy_power_graph()
```

In [10]:

```
def occupancy_power_graph():
    fig, ax1 = plt.subplots(1,1,figsize = (16, 4), dpi=800)
    data1 = deepcopy(data_dict["Living_room"]["raw"])
    data1.set_index(["time"],inplace=True)
    data1.index = pd.to_datetime(data1.index)
    data1["minOfDay"] = (data1.index.hour)*60+(data1.index.minute)
    df1 = data1.groupby(by=['minOfDay'])['occupancy presence (0/1)'].sum().reset_index()
    #print(df1)

    data2 = deepcopy(data_dict["Power"]["raw"])
    data2.set_index(["time"],inplace=True)
    data2.index = pd.to_datetime(data2.index)
    data2["minOfDay"] = (data2.index.hour)*60+(data2.index.minute)
    df2 = data2.groupby(by=['minOfDay'])['TV'].mean().reset_index()
    #print(df2)

    l1, = ax1.plot(df1["minOfDay"], df1["occupancy presence (0/1)"]/(365), linewidth=1, color="red")
    ax1.yaxis.set_major_locator(MaxNLocator(8,integer=True))
    ax1.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1, decimals=0))
    ax1.set_ylabel('Occupancy probability')
    ax1.set_xlabel('Time')
    ax2 = ax1.twinx()
    l2, = ax2.plot(df2["minOfDay"],df2["TV"],linewidth=1,color="grey")
    #ax2.set_xlabel('Time')
    ax2.set_ylabel('Average power (W)')
    ax2.set_xlim(0,1439)
    ax2.set_xticks([0,180,360,540,720,900,1080,1260])
    ax2.set_xticklabels(['00:00','03:00','06:00','09:00','12:00','15:00','18:00','21:00'])
    plt.legend(handles=[l1, l2], labels=['Living_room occupancy', 'TV'], loc=0)
    plt.savefig('occupancy and power.jpg', dpi=800, bbox_inches="tight",facecolor='w')
    #plt.show()
    plt.close()
occupancy_power_graph()
```

5. Peak power and total power consumption of appliances

In [11]:

```
app = ["electric kettle","fridge","rice cooker","computer","TV","washing machine","water heater","ac"]
app_colour_mapping = {"electric kettle":"darkorange",
                      "fridge":"gold",
                      "rice cooker":"yellow",
                      "computer":"greenyellow",
                      "TV":"green",
                      "washing machine":"blue",
                      "water heater":"deepskyblue",
                      "ac":"purple"}
app_mapping = {"electric kettle":"Electric kettle",
               "fridge":"Fridge",
               "rice cooker":"Rice cooker",
               "computer":"Computer",
               "TV":"TV",
               "washing machine":"Washing machine",
               "ac":"AC",
               "water heater":"Water heater"}
def app_peak_on_sum_graph():
    #fig, (ax1,ax2,ax3) = plt.subplots(3,1,figsize = (16,9), dpi=800)
    fig = plt.figure(figsize = (16,9), dpi=800)
    spec = fig.add_gridspec(nrows=3,ncols=1,height_ratios=[2,1,1])
    ax1 = fig.add_subplot(spec[0,0])
    ax2 = fig.add_subplot(spec[1,0])
    ax3 = fig.add_subplot(spec[2,0])

    data = deepcopy(data_dict["Power"]["raw"])

    df2 = data.set_index("time")
    df2 = df2.fillna(0)
    df2["water heater"] = df2["upstairs water heater"]+df2["downstairs water heater"]
    df2 = df2.drop(columns=["upstairs water heater","downstairs water heater"])
    df2["ac"] = df2["air conditioner#1"]+df2["air conditioner#2"]+df2["air conditioner#3"]+df2["air condi
    df2 = df2.drop(columns=["air conditioner#1","air conditioner#2","air conditioner#3","air conditioner#
    df2.loc['Col_sum'] = df2.apply(lambda x: x.sum())

    psum = df2.loc['Col_sum']
    index_sort = psum.sort_values(ascending=False).index
    sorted_data = df2[index_sort]
    df3 = sorted_data.rename(columns=app_mapping)
    x = [x.replace(' ', '\n') for x in df3.columns]
    y = (df3.loc['Col_sum'])/60000
    for i in range(len(x)):
        ax2.bar(x[i],y[i])
    for i in range(len(x)):
        ax3.bar(x[i],y[i])

    ax2.set_ylim(90, 2000)
    ax3.set_ylim(0, 40)

    ax2.spines['bottom'].set_visible(False)
    ax3.spines['top'].set_visible(False)
    ax2.xaxis.tick_top()
    ax2.set_xticks([])

    d = .85

    kwargs = dict(marker=[(-1, -d), (1, d)], markersize=15,
              linestyle='none', color='r', mec='r', mew=1, clip_on=False)
    ax2.plot([0, 1], [0, 0],transform=ax2.transAxes, **kwargs)
    ax3.plot([0, 1], [1, 1], transform=ax3.transAxes, **kwargs)

    del_index=[0,1,2,3,4]
    x_new_list=[i for num,i in enumerate(x) if num in del_index]
    y_new_list=[i for num,i in enumerate(y) if num in del_index]
    for a,b in zip(x_new_list,y_new_list):
        ax2.text(a,b,'%.2f'%b,ha='center',va='bottom');

    del_index=[0,1,2,3,4]
    x_new_list=[i for num,i in enumerate(x) if num not in del_index]
    y_new_list=[i for num,i in enumerate(y) if num not in del_index]
    for a,b in zip(x_new_list,y_new_list):
        ax3.text(a,b,'%.2f'%b,ha='center',va='bottom');

    df1 = data.drop(columns="time")
    df1["water heater"] = df1["upstairs water heater"]+df1["downstairs water heater"]
    df1 = df1.drop(columns=["upstairs water heater","downstairs water heater"])
```

```
    df1["ac"] = df1["air conditioner#1"]+df1["air conditioner#2"]+df1["air conditioner#3"]+df1["air condi
    df1 = df1.drop(columns=["air conditioner#1","air conditioner#2","air conditioner#3","air conditioner#
    index_sort = df1.max().sort_values(ascending=False).index
    df1 = df1[index_sort]
    df1 = df1.rename(columns=app_mapping)
    x = [x.replace(' ', '\n') for x in df1.columns]
    y = df1.max()
    for i in range(len(x)):
        ax1.bar(x[i],y[i])
    for a,b in zip(x,y):
        ax1.text(a,b,'%.2f'%b,ha='center',va='bottom');

    ax1.set_xlabel("(a) Peak power of appliances")
    ax3.set_xlabel("(b) Total power consumption of appliances")
    #fig.text(0.5, 0, 'Appliances', ha='center')
    fig.text(0.0, 0.8, "Peak power (W)", ha='center', va='center', rotation='vertical')
    fig.text(0.0, 0.32, "Power consumption (kWh)", ha='center', va='center', rotation='vertical')
    #fig.subplots_adjust(bottom=0.3)

    plt.tight_layout()
    fig.savefig('app_peak_on_sum.jpg', dpi=800, bbox_inches="tight",facecolor='w')
    #plt.show()
    plt.close()

app_peak_on_sum_graph()
```

In [12]:

```
def agg_power_graph():
    fig, (ax1,ax2) = plt.subplots(1,2,figsize = (16, 6), dpi=800)

    data = deepcopy(data_dict["Power"]["raw"])
    data["time"] = pd.to_datetime(data["time"])
    data["water heater"] = data["upstairs water heater"]+data["downstairs water heater"]
    data = data.drop(columns=["upstairs water heater","downstairs water heater"])
    data["ac"] = data["air conditioner#1"]+data["air conditioner#2"]+data["air conditioner#3"]+data["air
    data = data.drop(columns=["air conditioner#1","air conditioner#2","air conditioner#3","air conditioner

    # Agg power 20210801
    df_agg = data[(data["time"] >= pd.to_datetime('20210801')) & (data["time"] < pd.to_datetime('20210802
    df_agg = df_agg.set_index("time")
    #df_agg["total"]=df_agg.apply(lambda x: x.sum(),axis=1)
    #df_agg['Standby power'] = np.min(df_agg['total'])
    #print(df_agg)

    #cum = df_agg['Standby power']
    #ax1.fill_between(df_agg.index, np.zeros(df_agg.shape[0]), cum, label = 'Standby power', facecolor='di
    cum = np.zeros(df_agg.shape[0])
    for app_name in app:
        lower = cum
        cum = cum + df_agg[app_name]
        app_color = app_colour_mapping[app_name]
        ax1.fill_between(df_agg.index, lower, cum, label = app_mapping[app_name], facecolor=app_color)

    hours = mdates.HourLocator(interval = 4)
    h_fmt = mdates.DateFormatter('%H:%M')
    ax1.xaxis.tick_bottom()
    ax1.xaxis.set_major_locator(hours)
    ax1.xaxis.set_major_formatter(h_fmt)

    # Agg power 20210815
    df_agg1 = data[(data["time"] >= pd.to_datetime('20210815')) & (data["time"] < pd.to_datetime('2021081
    df_agg1 = df_agg1.set_index("time")
    #df_agg1["total"]=df_agg1.apply(lambda x: x.sum(),axis=1)
    #df_agg1['Standby power'] = np.min(df_agg1['total'])
    #print(df_agg1)

    #cum1 = df_agg1['Standby power']
    #ax2.fill_between(df_agg1.index, np.zeros(df_agg1.shape[0]), cum1, label = 'Standby power', facecolor=
    cum1 = np.zeros(df_agg.shape[0])
    for app_name in app:
        lower1 = cum1
        cum1 = cum1 + df_agg1[app_name]
        app_color = app_colour_mapping[app_name]
        ax2.fill_between(df_agg1.index, lower1, cum1, label = app_mapping[app_name], facecolor=app_color)

    hours = mdates.HourLocator(interval = 4)
    h_fmt = mdates.DateFormatter('%H:%M')
    ax2.xaxis.tick_bottom()
    ax2.xaxis.set_major_locator(hours)
    ax2.xaxis.set_major_formatter(h_fmt)

    handles, labels = ax1.get_legend_handles_labels()
    fig.legend(handles, labels, loc='lower center', bbox_to_anchor=(0.5, 0), borderaxespad=0, ncol=4)

    ax1.grid(b=True, which='major', color='#999999', linestyle='--', alpha = 0.1)
    ax2.grid(b=True, which='major', color='#999999', linestyle='--', alpha = 0.1)
    ax1.minorticks_on()
    ax2.minorticks_on()
    ax1.grid(b=True, which='minor', color='#999999', linestyle='--', alpha = 0.2)
    ax2.grid(b=True, which='minor', color='#999999', linestyle='--', alpha = 0.2)
    ax1.set_ylim([0,5000])
    ax2.set_ylim([0,5000])

    ax1.set_title('2021-08-01')
    ax2.set_title('2021-08-15')

    fig.text(0.5, 0.2, 'Time', ha='center')
    fig.text(0.07, 0.65, 'Power (W)', ha='center', va='center', rotation='vertical')
    fig.subplots_adjust(bottom=0.3)

    fig.savefig('agg_power.jpg', dpi=800, bbox_inches="tight",facecolor='w')
    #plt.show()
    plt.close()
```

```
77
78 agg_power_graph()
```

6. Whole-year electricity consumption of appliances

In [13]:
```
1  def ac_graph():
2      fig, ax1 = plt.subplots(1,1,figsize = (16,4), dpi=800)
3
4      x = list(range(0,24))
5      y = [0.03,0.29,0.38,0.48,0.57,0.67,0.67,0.76,0.86,0.95,1.12,1.14,1.33,1.52,1.81,1.81,2,2.19,
6          3.24,3.52,5.71,6.67,9.9,14.15]
7
8      ax1.bar(x,y, color=["grey","grey","grey","grey","grey","grey","grey","grey","grey","grey","red",
9                          "grey","grey","grey","grey","grey","grey","grey","grey","grey","grey","grey",
10                         "grey","grey"])
11     for a,b in zip(x,y):
12         ax1.text(a,b,'%.2f'%b,ha='center',va='bottom');
13
14     ax1.set_ylabel("AC electricity consumption\n" + "(kWh/(m\u00b2•a)")
15     ax1.yaxis.set_major_locator(MaxNLocator(8,integer=True))
16     plt.xticks(x,('1',' ','3',' ','5',' ','7',' ','9',' ','CN-OBEE',' ','12',' ','14',' ','16',
17                 ' ','18',' ','20',' ','22',' '))
18     plt.tight_layout()
19     fig.savefig('ac.jpg', dpi=800, bbox_inches="tight",facecolor='w')
20     #plt.show()
21     plt.close()
22
23 ac_graph()
```

In [ ]:
```
1
```